

LangGraph Nodes

A Guide to State Manipulation and Execution

In LangGraph, a graph is composed of state, nodes, and edges. Nodes are the fundamental units of computation. This document details how nodes are defined, how they interact with the graph state, and provides literal code execution examples.

1. The Graph State

Before defining a node, the underlying **State** must be established. The state is typically defined using a **TypedDict**. Each node receives this state as input and returns a dictionary containing updates to the state.

```
from typing import TypedDict, Annotated, Sequence
import operator
from langchain_core.messages import BaseMessage

class AgentState(TypedDict):
    messages: Annotated[Sequence[BaseMessage], operator.add]
    context: str
```

2. Defining a Node

A node is literally a Python function or a LangChain **Runnable** (such as a chain or an LLM call). The function signature takes the current state (a dictionary) and returns an updated dictionary.

Standard Python Function Node

This node takes the state, processes it, and returns the key-value pairs that need to be appended or overridden in the state.

```
def fetch_context_node(state: AgentState):
    # Literal execution: extract messages, perform logic, return state update
    messages = state.get("messages", [])

    # ... logic to fetch context based on the last message ...
    fetched_context = "System documentation regarding LangGraph."

    # Return exactly what needs to be updated in the state
    return {"context": fetched_context}
```

LLM/Runnable Node

Nodes often encapsulate an LLM call. You invoke the model within the node function and return the resulting message.

```
from langchain_core.messages import AIMessage
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4")

def agent_node(state: AgentState):
    messages = state["messages"]
    context = state.get("context", "")

    # Format system prompt with context if available
    system_msg = {"role": "system", "content": f"Use context: {context}"}
    invoke_messages = [system_msg] + messages

    # Execute LLM call
    response = llm.invoke(invoke_messages)

    # Reducer (operator.add) will append this AIMessage to the list
    return {"messages": [response]}
```

3. Adding Nodes to the Graph

Nodes are registered to a `StateGraph` using the `add_node` method. You provide a unique string identifier and the function/runnable.

```
from langgraph.graph import StateGraph

# Initialize graph with the schema
workflow = StateGraph(AgentState)

# Add standard nodes
workflow.add_node("fetch_context", fetch_context_node)
workflow.add_node("agent", agent_node)
```

4. Special Nodes (START and END)

LangGraph uses virtual nodes to define entry and exit points for the execution loop.

- **START**: The initial entry point of the graph. You draw an edge from **START** to the first executing node.
- **END**: The terminal point. Drawing an edge to **END** halts the graph's execution.

```
from langgraph.graph import START, END

# Define entry point
workflow.add_edge(START, "fetch_context")

# Define flow between nodes
workflow.add_edge("fetch_context", "agent")

# Define exit point
workflow.add_edge("agent", END)
```

5. Tool Calling Nodes (prebuilt.ToolNode)

When executing tool calls, LangGraph provides a prebuilt **ToolNode** that automatically extracts tool invocations from the **AIMessage**, executes the bound tools, and returns **ToolMessage** objects to the state.

```
from langgraph.prebuilt import ToolNode
from langchain_core.tools import tool

@tool
def get_weather(location: str) -> str:
    """Get weather for a location."""
    return "Sunny and 75F"

tools = [get_weather]
tool_node = ToolNode(tools)

# Add to graph
workflow.add_node("tools", tool_node)
```

Execution Detail: When a node executes, it only needs to return the keys defined in the `State` schema that have changed. If a key uses a reducer (like `operator.add`), the returned value is combined with the existing value. If no reducer is specified, the returned value completely overwrites the existing value.